

ECE 60146 HW 07

Andrea Goh

March 16, 2025

1 Extract Multi-instance Dataset from COCO

In this assignment, the **COCO 2014 Train images** and **2014 Test images** were used to generate a custom Multi-Instance Object Localization Dataset. The criteria for each image in the custom dataset must contain at least one foreground object from one of the three categories: ['pizza', 'cat', 'bus']. The area of any foreground object must be larger than 40,000 pixels.

1.1 Custom Dataset Example



Figure 1: Training Dataset Examples

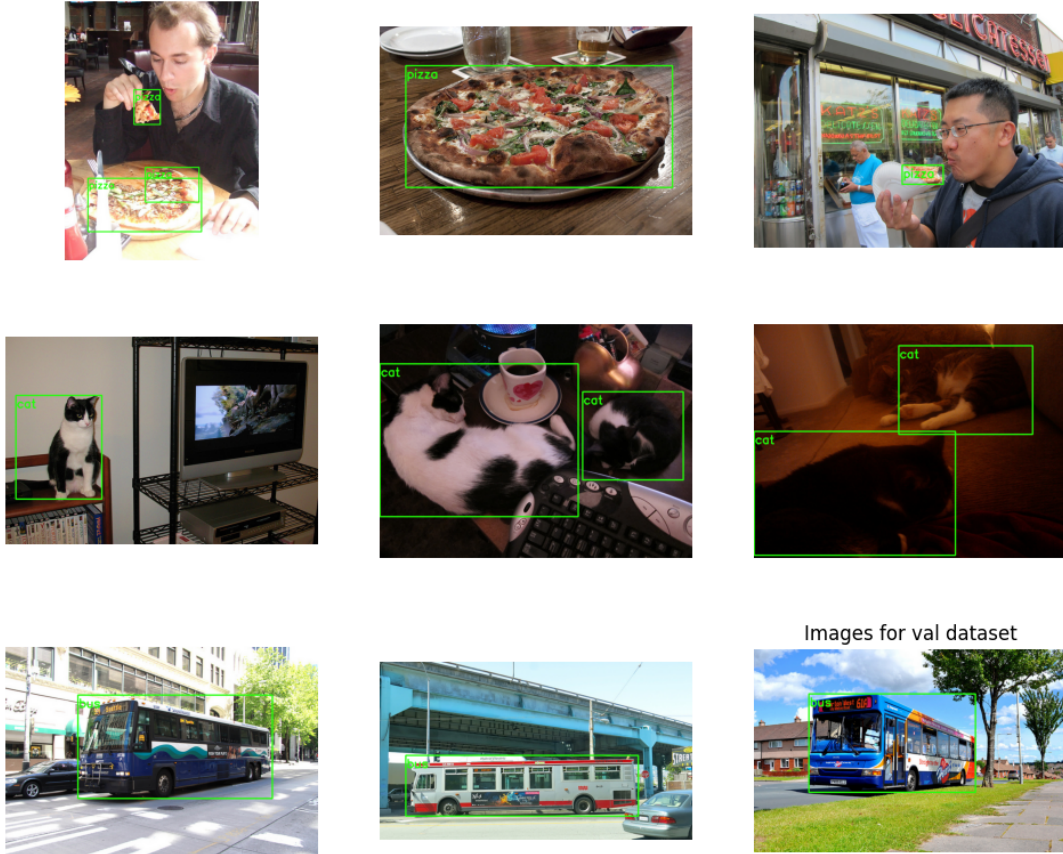


Figure 2: Validation Dataset Examples

1.2 Custom Dataset Code

```

1 from pycocotools.coco import COCO
2 import numpy as np
3 import cv2
4 import matplotlib.pyplot as plt
5 import json
6 import sys
7 import random
8
9 def generate_dataset(type):
10     input_json = f'annotations/instances_{type}2014.json'
11     class_list = ['pizza', 'cat', 'bus']
12
13     # Mapping from COCO label to class indices
14     coco_labels_inverse = {}
15     coco = COCO(input_json)
16     catIds = coco.getCatIds(catNms=class_list)
17     categories = coco.loadCats(catIds)
18     categories.sort(key=lambda x: x['id'])
19
20     for idx, in_class in enumerate(class_list):
21         for c in categories:
22             if c['name'] == in_class:
23                 coco_labels_inverse[c['id']] = idx
24
25     imgIds = set() # use set to avoid duplication of images
26     # Retrieve Image list
27     for catId in catIds:
28         imgIds.update(coco.getImgIds(catIds=catId))
29
30     filtered_img_ids = [] # Stores valid image IDs
31     for img_id in imgIds:
32         ann_ids = coco.getAnnIds(imgIds=img_id, iscrowd=False)

```

```

33     anns = coco.loadAnns(ann_ids)
34
35     # Get unique categories in the image
36     obj_counts = {}
37     for ann in anns:
38         obj_category = coco.loadCats(ann['category_id'])[0]['name']
39         obj_counts[obj_category] = obj_counts.get(obj_category, 0) + 1
40
41     # Check if the image contains more than one type of object
42     if len(obj_counts) > 1:
43         # Check for large foreground objects
44         has_large_foreground = False
45         for ann in anns:
46             area = ann['area'] # COCO format: [x_min, y_min, width, height]
47             if area >= 40000: # Check if area is greater than 40,000 pixels
48                 has_large_foreground = True
49                 break # No need to check further, found a valid object
50
51         if has_large_foreground:
52             filtered_img_ids.append(img_id)
53
54     print(f"Filtered_{len(filtered_img_ids)}_{type} images that meet the criteria.")
55
56     filtered_images = [img for img in coco.dataset["images"] if img["id"] in
57                        filtered_img_ids]
58
59     # Filter annotations
60     filtered_annotations = [ann for ann in coco.dataset["annotations"] if ann["image_id"]
61                            ] in filtered_img_ids]
62
63     # Filter categories (only include used categories)
64     used_category_ids = {ann["category_id"] for ann in filtered_annotations}
65     filtered_categories = [cat for cat in coco.dataset["categories"] if cat["id"] in
66                           used_category_ids]
67
68     # New COCO dataset structure
69     filtered_coco_data = {
70         "images": filtered_images,
71         "annotations": filtered_annotations,
72         "categories": filtered_categories
73     }
74
75     # Save the new JSON file
76     with open(f"annotations/filtered_{type}2014.json", "w") as f:
77         json.dump(filtered_coco_data, f, indent=4)
78
79     print(f"Filtered_dataset_saved_as_annotations/filtered_{type}2014.json")
80
81 def plot(type):
82     input_json = f'annotations/filtered_{type}2014.json'
83     class_list = ['pizza', 'cat', 'bus']
84
85     # Load the filtered dataset
86     with open(input_json, "r") as f:
87         coco_data = json.load(f)
88
89     # Convert data to COCO format
90     coco = COCO(input_json)
91
92     # Create a mapping from category names to category IDs
93     cat_name_to_id = {cat["name"]: cat["id"] for cat in coco_data["categories"]}
94
95     # Get three images per category
96     fig, axes = plt.subplots(len(class_list), 3, figsize=(12, 10))
97
98     for row, class_name in enumerate(class_list):
99         cat_id = cat_name_to_id[class_name]
100         img_ids = coco.getImgIds(catIds=[cat_id])
101
102         # Select up to three images
103         selected_img_ids = [img_ids[i] for i in random.sample(range(len(img_ids)), 3)]
104
105         for col, img_id in enumerate(selected_img_ids):

```

```

103         img_info = coco.loadImgs(img_id)[0]
104         img = cv2.imread(f"{type}2014/{img_info['file_name']}")
105         img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
106
107         ann_ids = coco.getAnnIds(imgIds=img_id, catIds=[cat_id])
108         anns = coco.loadAnns(ann_ids)
109
110         # Draw bounding boxes
111         for ann in anns:
112             x, y, w, h = ann["bbox"]
113             cv2.rectangle(img, (int(x), int(y)), (int(x + w), int(y + h)), (36, 255,
114                                     12), 2)
115             cv2.putText(img, f'{class_name}', (int(x+2), int(y+25)), cv2.
116                         FONT_HERSHEY_SIMPLEX, 0.8, (36, 255, 12), 2)
117
118         axes[row, col].imshow(img)
119         axes[row, col].axis("off")
120     plt.title(f'Images_{for_{type}_{dataset}}')
121     plt.savefig(f'{type}_{dataset}.png', bbox_inches='tight')
122
123 if __name__ == '__main__':
124     if len(sys.argv) == 2:
125         generate_dataset(sys.argv[1])
126         # python3 generate_dataset.py train
127         # python3 generate_dataset.py val
128     else:
129         plot(sys.argv[2])
130         # python3 generate_dataset.py plot train
131         # python3 generate_dataset.py plot val

```

2 Dataloader

The **MyDataset** class loads the images and their corresponding annotations from the filtered json files generated in section 1. The class inherits from `torch.utils.data.Dataset` so that the dataset can be wrapped with PyTorch's **DataLoader**.

2.1 Code

```

1 class MyDataset(Dataset):
2     def __init__(self, ann_file, img_dir, S=8, datatype='train'):
3         self.img_dir = img_dir
4         self.S = S
5         self.coco = COCO(ann_file)
6         self.class_list = ['pizza', 'cat', 'bus']
7         self.cat_id = self.coco.getCatIds(catNms=self.class_list)
8         imgIds = []
9         for catId in self.cat_id: # Retrieve Image list
10             imgIds.extend(self.coco.getImgIds(catIds=catId))
11         self.img_ids = imgIds
12         self.aspect = [1/5, 1/3, 1/1, 3/1, 5/1]
13         self.datatype = datatype
14
15     def __len__(self):
16         return len(self.img_ids)
17
18     def __getitem__(self, idx):
19         img_info = self.coco.loadImgs(self.img_ids[idx])[0]
20         H = img_info['height']
21         W = img_info['width']
22         img = Image.open(f"{self.img_dir}/{img_info['file_name']}")
23         ann_ids = self.coco.getAnnIds(imgIds=img_info['id'], catIds=self.cat_id, iscrowd
24                                     =None)
25         anns = self.coco.loadAnns(ann_ids)
26
27         yolo_tensor = torch.zeros((self.S, self.S, 5, 5 + len(self.cat_id))) #
28             Initialize yolo_tensor
29         for ann in anns:
30             x, y, w, h = ann['bbox'] # Get bbox from annotations

```

```

30         # Get normalized center coordinates of bbox
31         x0 = (x + w/2)/W
32         y0 = (y + h/2)/H
33         # Find grid cell closest to center of bbox
34         x_grid = int(x0*self.S)
35         y_grid = int(y0*self.S) # idx 0-7
36         # Get displacement of bbox and center of cell
37         x_diff = (x0 - (x_grid+0.5)/self.S) * 256/8
38         y_diff = (y0 - (y_grid+0.5)/self.S) * 256/8
39         # Find index of anchor box with aspect ratio closest to bbox aspect
40         ratio = h/w
41         anchor_inx = min(range(len(self.aspect)), key=lambda i: abs(self.aspect[i] -
                                ratio))
42
43         yolo_vector = torch.zeros((5 + len(self.cat_id))) # Initialize yolo_vector
44         yolo_vector[:5] = torch.tensor([1, x_diff, y_diff, h/H, w/W])
45         yolo_vector[5:] = torch.tensor([1 if id == ann['category_id'] else 0 for id
                                in self.cat_id])
46         yolo_tensor[y_grid, x_grid, anchor_inx, :] = yolo_vector
47     if self.datatype=='train' or self.datatype=='val':
48         transform = transforms.Compose([
49             transforms.Grayscale(num_output_channels=3),
50             transforms.Resize((256,256)),
51             transforms.ToTensor(),
52             transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
53         ])
54         img = transform(img)
55     elif self.datatype=='disp':
56         transform = transforms.Compose([
57             transforms.Grayscale(num_output_channels=3),
58             transforms.Resize((256,256))
59         ])
60         img = transform(img)
61     return img, yolo_tensor

```

2.2 Dataloader Code Explanation

1. Initialization (`__init__`):

The class is initialized with the annotation file(**ann_file**), the directory where the images are stored (**img_dir**), the grid size (**S**), and the dataset type.

2. `__len__` Method:

This method returns the total number of images in the dataset, based on the length of the image ID list defined in the initialization.

3. `__getitem__` Method:

Given an index **idx**, the image and the corresponding annotations are retrieved. If the datatype is specified as either "train" or "val", the image is downsized to (256, 256), converted into a tensor, and normalized, transforming the image to a suitable format for training and testing the model. If the datatype is specified as "dis", the image is simply downsized to (256,256), which makes the image suitable for displaying purposes.

For each annotation in the image, the bounding box is extracted: [top left x position, top left y position, width, height]. The center of each bounding box is mapped to a specific cell in a $S \times S$ grid. The displacement from the bounding box center to the center of that cell is calculated (δx and δy). Finally, an anchor box is selected based on the aspect ratio of the bounding box. Then, the **yolo_vector** is constructed: $[1, \delta x, \delta y, h, w, 1, 0, 0]$, where 1 indicates the presence of a bounding box, h and w represent the height and width of the bounding box normalized by the image dimensions, and the last three elements correspond to the one-hot encoding of the object class. This **yolo_vector** is placed into the **yolo_tensor** in the position $[x_grid, y_grid, anchor_idx, :]$, where x_grid and y_grid represent the grid cell that was assigned and the $anchor_idx$ represent the anchor box that has the closest aspect ratio to the bounding box. For any grid cell and anchor boxes that do not contain the center of the bounding box, the yolo vector is all zeros: $[0, 0, 0, 0, 0, 0, 0, 0]$.

3 Building The Deep Neural Network

The Deep Neural Network used in this assignment was borrowed directly from the YOLOLogic platform: <https://engineering.purdue.edu/kak/distYOLO/>. The only changes made were in the 71 to 79 in the code block shown below.

3.1 Code

```
1 class SkipBlock(nn.Module):
2     """
3     This is a building-block class that I have borrowed from the YOLOLogic platform
4     """
5     def __init__(self, in_ch, out_ch, downsample=False, skip_connections=True):
6         super(SkipBlock, self).__init__()
7         self.downsample = downsample
8         self.skip_connections = skip_connections
9         self.in_ch = in_ch
10        self.out_ch = out_ch
11        self.conv1 = nn.Conv2d(in_ch, in_ch, 3, stride=1, padding=1)
12        self.conv2 = nn.Conv2d(in_ch, out_ch, 3, stride=1, padding=1)
13        self.bn1 = nn.BatchNorm2d(in_ch)
14        self.bn2 = nn.BatchNorm2d(out_ch)
15        self.in2out = nn.Conv2d(in_ch, out_ch, 1)
16        if downsample:
17            ## Setting stride to 2 and kernel_size to 1 amounts to retaining every
18            ## other pixel in the image --- which halves the size of the image:
19            self.downsampler1 = nn.Conv2d(in_ch, in_ch, 1, stride=2)
20            self.downsampler2 = nn.Conv2d(out_ch, out_ch, 1, stride=2)
21
22        def forward(self, x):
23            identity = x
24            out = self.conv1(x)
25            out = self.bn1(out)
26            out = nn.functional.relu(out)
27            out = self.conv2(out)
28            out = self.bn2(out)
29            out = nn.functional.relu(out)
30            if self.downsample:
31                identity = self.downsampler1(identity)
32                out = self.downsampler2(out)
33            if self.skip_connections:
34                if (self.in_ch == self.out_ch) and (self.downsample is False):
35                    out = out + identity
36                elif (self.in_ch != self.out_ch) and (self.downsample is False):
37                    identity = self.in2out( identity )
38                    out = out + identity
39                elif (self.in_ch != self.out_ch) and (self.downsample is True):
40                    out = out + torch.cat((identity, identity), dim=1)
41            return out
42
43 class NetForYolo(nn.Module):
44     """
45     This is a building-block class that I have borrowed from the YOLOLogic platform
46     """
47     def __init__(self, skip_connections=True, depth=8):
48         super(NetForYolo, self).__init__()
49         self.depth = depth // 2
50         self.conv1 = nn.Conv2d(3, 64, 3, padding=1)
51         self.conv2 = nn.Conv2d(64, 64, 3, padding=1)
52         self.pool = nn.MaxPool2d(2, 2)
53         self.bn1 = nn.BatchNorm2d(64)
54         self.bn2 = nn.BatchNorm2d(128)
55         self.bn3 = nn.BatchNorm2d(256)
56         self.skip64_arr = nn.ModuleList()
57         for i in range(self.depth):
58             self.skip64_arr.append(SkipBlock(64, 64, skip_connections=skip_connections))
59         self.skip64ds = SkipBlock(64,64,downsample=True, skip_connections=
            skip_connections)
60         self.skip64to128 = SkipBlock(64, 128, skip_connections=skip_connections )
61         self.skip128_arr = nn.ModuleList()
62         for i in range(self.depth):
```

```

63         self.skip128_arr.append(SkipBlock(128,128, skip_connections=skip_connections
64         ))
65     self.skip128ds = SkipBlock(128,128, downsample=True, skip_connections=
66     skip_connections)
67     self.skip128to256 = SkipBlock(128, 256, skip_connections=skip_connections )
68     self.skip256_arr = nn.ModuleList()
69     for i in range(self.depth):
70         self.skip256_arr.append(SkipBlock(256,256, skip_connections=skip_connections
71         ))
72     self.skip256ds = SkipBlock(256,256, downsample=True, skip_connections=
73     skip_connections)
74     self.fc_seqn = nn.Sequential(
75         nn.Linear(8192, 4096),
76         nn.ReLU(inplace=True),
77         nn.Linear(4096, 2048),
78         nn.ReLU(inplace=True),
79         nn.Linear(2048, 1024),
80         nn.ReLU(inplace=True),
81         nn.Linear(1024, 640)
82     )
83
84     def forward(self, x):
85         x = self.pool(torch.nn.functional.relu(self.conv1(x)))
86         x = nn.MaxPool2d(2,2)(torch.nn.functional.relu(self.conv2(x)))
87         for i,skip64 in enumerate(self.skip64_arr[:self.depth//4]):
88             x = skip64(x)
89         x = self.skip64ds(x)
90         for i,skip64 in enumerate(self.skip64_arr[self.depth//4:]):
91             x = skip64(x)
92         x = self.bn1(x)
93         x = self.skip64to128(x)
94         for i,skip128 in enumerate(self.skip128_arr[:self.depth//4]):
95             x = skip128(x)
96         x = self.bn2(x)
97         x = self.skip128ds(x)
98         x = x.view(-1, 8192)
99         x = self.fc_seqn(x)
100        return x

```

4 Training

Three loss functions are used to train the YOLO model:

1. Binary Cross-Entropy Loss

BCE loss is used for object detection. PyTorch's **torch.nn.BCEWithLogitsLoss(pos_weight)** is used, which allows us to weigh false negatives more heavily than the baseline **torch.nn.BCELoss**. Each image has 8×8 grid cells, with 5 anchor boxes assigned to each cell. Therefore, there are a total of $8 \times 8 \times 5 = 320$ yolo vectors for each image. Assuming that most of the images contain single instances from the classes ['pizza', 'cat', 'bus'], there will be 319 yolo vectors of negatives and 1 positive yolo vector. Thus, the pos.weight is set to 319, which means a false negative is weighted 319 times more than a false positive in the loss function. This weight prevents the model from predicting all negatives due to the imbalance of positive and negative samples.

$$loss = -\log(p) - 319 \log(1 - p) \quad (1)$$

2. Mean Squared Error Loss

MSE loss is used for bounding box:

$$MSE = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2 \quad (2)$$

3. Cross Entropy Loss

Cross entropy loss is used for classification:

$$CE = - \sum_{c=1}^M y_c \log(p_c) \quad (3)$$

4.1 Code

```
1 def plot_loss(loss_tally, title, color):
2     plt.figure(figsize=(10,5))
3     plt.plot(loss_tally, label=title, color=color)
4     plt.xlabel("Iterations")
5     plt.ylabel("Loss")
6     plt.title(f"{title}_vs._Iterations")
7     plt.legend()
8     plt.savefig(f"{title}.png")
9     plt.cla()
10    return
11
12 def run_code_for_training(model, loader, device):
13     learning_rate = 0.0001
14     model.train()
15     pos_weight = torch.tensor([319.0]) # Balancing Factor approximate
16     criterion1 = nn.BCEWithLogitsLoss(pos_weight = pos_weight).to(device) # for element
17     # in position 0 (detection)
18     criterion2 = nn.MSELoss() # for elements in position 1,2,3,4 (bbox)
19     criterion3 = nn.CrossEntropyLoss() # for elements in position 5,6,7 (classification)
20     print("\n\nLearning_Rate:", learning_rate)
21     optim = torch.optim.Adam(model.parameters(), lr=learning_rate, betas=(0.9,0.99))
22     print("\n\nStarting_training_loop...\n\n")
23     start_time = time.perf_counter()
24     epochs = 15
25     obj_loss_tally = []
26     bbox_loss_tally = []
27     class_labeling_loss_tally = []
28
29     for epoch in range(epochs):
30         print('')
31         running_loss = [0.0, 0.0, 0.0]
32         for iter, data in enumerate(loader):
33             img_tensor, yolo_tensor = data
34
35             img_tensor = img_tensor.to(device)
36             yolo_tensor = yolo_tensor.to(device)
37             optim.zero_grad()
38             pred = model(img_tensor)
39             pred = pred.view(-1,8,8,5,8) # 8,8,8,5,8
40             pred[:, :, :, :, 0] = (pred[:, :, :, :, 0])
41             loss = torch.tensor(0.0, requires_grad=True).float().to(device)
42
43             ## Compute object mask
44             mask = yolo_tensor[:, :, :, :, 0] == 1
45             indices = mask.nonzero(as_tuple=True)
46             yolo_tensor_mask = yolo_tensor[indices]
47             pred_mask = pred[indices] # sigmoid or no sigmoid
48
49             ## Estimating presence/absence of object with the Binary Cross Entropy loss
50             :
51             obj_loss = criterion1((pred[:, :, :, :, 0]), yolo_tensor[:, :, :, :, 0])
52
53             ## MSE loss for the regression params for the object bounding boxes:
54             bbox_loss = criterion2(pred_mask[:, 1:5], yolo_tensor_mask[:, 1:5])
55
56             ## CrossEntropy loss for object class labels
57             class_labeling_loss = criterion3(pred_mask[:, 5:], yolo_tensor_mask[:, 5:])
58
59             # Adjust loss weights
60             loss = obj_loss + bbox_loss + class_labeling_loss
61             loss.backward()
62             optim.step()
63
64             # Update running loss
65             running_loss[0] += obj_loss.item()
66             running_loss[1] += bbox_loss.item()
67             running_loss[2] += class_labeling_loss.item()
68
69             if iter%100==99:
70                 current_time = time.perf_counter()
71                 elapsed_time = current_time - start_time
```

```

70     avg_loss = [x/100 for x in running_loss]
71     print("\n[epoch:%d/%d, iter=%4d, elapsed_time=%5d, secs] obj_loss: %7.4f %7.4f %7.4f" %
72           (epoch+1, epochs, iter+1, elapsed_time, avg_loss[0],
73            avg_loss[1], avg_loss[2]))
74     obj_loss_tally.append(avg_loss[0])
75     bbox_loss_tally.append(avg_loss[1])
76     class_labeling_loss_tally.append(avg_loss[2])
77     running_loss = [0.0, 0.0, 0.0]
78
79     print("\nFinished Training\n")
80     torch.save(model.state_dict(), 'train_model')
81     plot_loss(obj_loss_tally, 'Object Loss (BCE)', 'blue')
82     plot_loss(bbox_loss_tally, 'Bounding Box Loss (MSE)', 'orange')
83     plot_loss(class_labeling_loss_tally, 'Class Labeling Loss (CrossEntropy)', 'green')
84     # Combined loss plot
85     plt.figure(figsize=(10,5))
86     plt.plot(obj_loss_tally, label="Object Loss (BCE)")
87     plt.plot(bbox_loss_tally, label="Bounding Box Loss (MSE)", color='orange')
88     plt.plot(class_labeling_loss_tally, label="Class Labeling Loss (CrossEntropy)",
89              color='green')
90     plt.xlabel("Iterations")
91     plt.ylabel("Loss")
92     plt.title("Loss Comparison")
93     plt.legend()
94     plt.savefig("combined_loss.png")
95     plt.cla()
96     return model

```

4.2 Hyperparameters

Grid Size	8×8
Anchor Boxes	5
Batch Size	8
Epochs	15
Optimizer	Adam
Learning Rate	0.0001

Table 1: Hyperparameters

4.3 Results



Figure 3: Object Loss

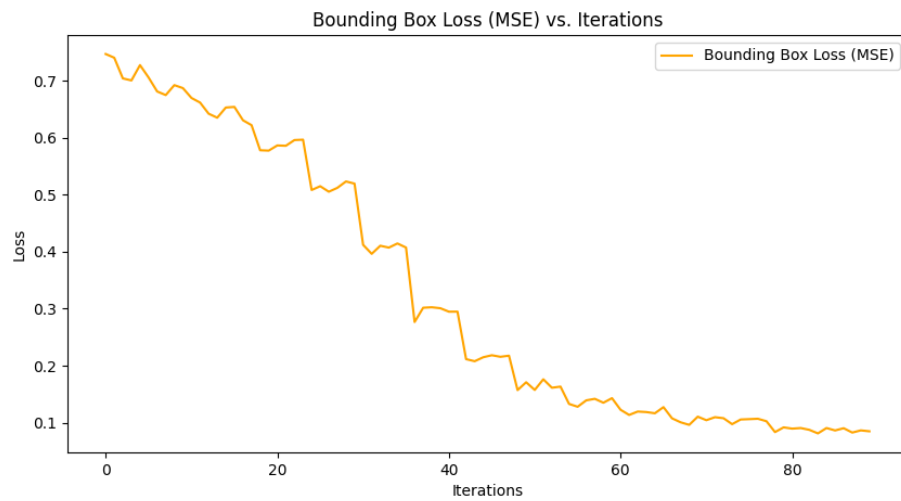


Figure 4: Bounding Box Loss

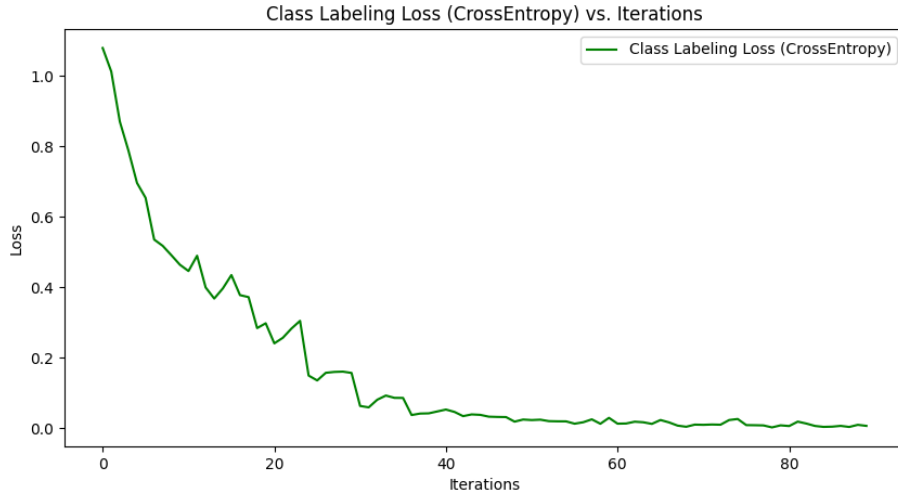


Figure 5: Class Labeling Loss

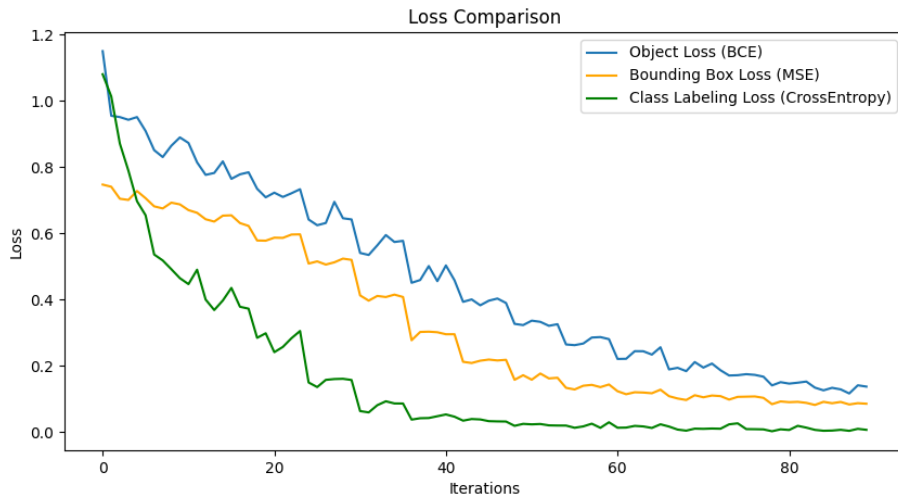


Figure 6: Combined Losses

5 Evaluation

5.1 Code

```

1 def run_code_for_testing(model, device):
2     class_list = ['bus', 'cat', 'pizza']
3     model.load_state_dict(torch.load('train_model'))
4     model = model.to(device)
5     testDataset = MyDataset('annotations/filtered_val2014.json', 'val2014', S=8,
6                             datatype='val')
7     testDataLoader = DataLoader(testDataset, shuffle=False, batch_size = 8)
8     dispDataset = MyDataset('annotations/filtered_val2014.json', 'val2014', S=8,
9                             datatype='disp')
10    class_correct = [0] * 3
11    class_total = [0] * 3
12    iou_list = []
13    gt_labels = []
14    pred_labels = []
15    conf_threshold = 0.7 # Confidence of detection
16    iou_threshold = 0.2 # If overlap is more than 0.2, discard bbox

```

```

15
16 with torch.no_grad():
17     for iter,data in enumerate(testDataLoader):
18
19         im_tensor, gt_yolo_tensor = data
20         im_tensor = im_tensor.to(device)
21         gt_yolo_tensor = gt_yolo_tensor.to(device)
22         output = model(im_tensor)
23         pred_yolo_tensor = output.view(-1, 8, 8, 5, 8)
24         pred_yolo_tensor[:, :, :, :, 0] /= torch.max(pred_yolo_tensor[:, :, :, :,
25             0])
26         batchsize = len(pred_yolo_tensor)
27         for img in range(batchsize): # for each img in batch of 8
28             # Get ground truth bboxes and classes
29             gt_mask = gt_yolo_tensor[img,:,:,:,0] == 1
30             gt_indices = gt_mask.nonzero()
31             gt_bboxes = []
32             gt_class_idxes = []
33             gt_classes = []
34             for gt_index in gt_indices: # for each detected yolo vector
35                 gt_yolo_vector = gt_yolo_tensor[img, gt_index[0], gt_index[1],
36                     gt_index[2],:]
37                 gt_bboxes.append(get_bbox(gt_index, gt_yolo_vector))
38                 gt_class_idx = torch.argmax(gt_yolo_vector[5:8]).item()
39                 gt_class_idxes.append(gt_class_idx)
40                 gt_classes.append(class_list[gt_class_idx])
41
42             # Get predicted bboxes and classes using NMS
43             pred_mask = pred_yolo_tensor[img, :, :, :, 0] > conf_threshold
44             pred_indices = pred_mask.nonzero()
45             bboxes = []
46             scores = []
47             labels = []
48             for pred_index in pred_indices:
49                 pred_yolo_vector = pred_yolo_tensor[img, pred_index[0], pred_index
50                     [1], pred_index[2], :]
51                 pred_bbox = get_bbox(pred_index, pred_yolo_vector)
52                 score = pred_yolo_vector[0].item() # Confidence score
53                 pred_class_idx = torch.argmax(pred_yolo_vector[5:8]).item()
54                 bboxes.append(pred_bbox)
55                 scores.append(score)
56                 labels.append(pred_class_idx)
57             # Convert lists to tensors
58             bboxes = torch.tensor(bboxes, dtype=torch.float32).to(device)
59             scores = torch.tensor(scores, dtype=torch.float32).to(device)
60             labels = torch.tensor(labels, dtype=torch.int64).to(device)
61             pred_bboxes = []
62             pred_class_idxes = []
63             pred_classes = []
64
65             if len(pred_indices) != 0:
66                 # Apply NMS
67                 keep_indices = ops.nms(bboxes, scores, iou_threshold)
68                 nms_bboxes = bboxes[keep_indices]
69                 nms_labels = labels[keep_indices]
70                 # Store final detections
71                 for i in range(len(keep_indices)):
72                     pred_bboxes.append(nms_bboxes[i].tolist())
73                     pred_class_idx = nms_labels[i]
74                     pred_class_idxes.append(pred_class_idx)
75                     pred_classes.append(class_list[nms_labels[i]])
76             else:
77                 max_conf_index = torch.argmax(pred_yolo_tensor[img, :, :, :, 0]) #
78                 # Index of the highest confidence score
79                 max_conf_index = torch.unravel_index(max_conf_index,
80                     pred_yolo_tensor[img, :, :, :, 0].shape)
81                 pred_yolo_vector = pred_yolo_tensor[img,max_conf_index[0].item(),
82                     max_conf_index[1],max_conf_index[2]]
83                 pred_bbox = get_bbox(max_conf_index, pred_yolo_vector)
84                 pred_bboxes.append(pred_bbox)
85                 pred_class_idx = torch.argmax(pred_yolo_vector[5:8]).item()
86                 pred_class_idxes.append(pred_class_idx)
87                 pred_classes.append(class_list[pred_class_idx])

```

```

82         iou = IoU_calculator(gt_bboxes, pred_bboxes)
83         iou_list.append(iou)
84         class_total[gt_class_idxes[0]] += 1
85         gt_labels.append(gt_class_idxes[0])
86         pred_labels.append(pred_class_idxes[0])
87         if gt_class_idxes[0] == pred_class_idxes[0]:
88             class_correct[gt_class_idxes[0]] += 1
89         gt_class = gt_classes[0]
90         pred_class = pred_classes[0]
91         root = f'Results/{gt_class}_{pred_class}'
92         iou_thresholds = [(0.9, 'results_iou>0.9'), (0.8, 'results_iou>0.8'),
93             , (0.7, 'results_iou>0.7'),
94                 (0.6, 'results_iou>0.6'), (0.5, 'results_iou>0.5'),
95                 (0.4, 'results_iou>0.4'),
96                 (0.3, 'results_iou>0.3'), (0.2, 'results_iou>0.2'),
97                 (0.1, 'results_iou>0.1'),
98                 (0.0, 'results_iou>0.0')]
99         for threshold, label in iou_thresholds:
100             if iou >= threshold:
101                 root = os.path.join(root, label)
102                 break
103         display_img(disDataset, gt_bboxes, gt_classes, pred_bboxes,
104             pred_classes, iter*8+img, root)
105     # Plot confusion matrix
106     cm = confusion_matrix(torch.tensor(gt_labels).cpu().numpy(), torch.tensor(
107         pred_labels).cpu().numpy())
108     plt.figure(figsize=(6, 6))
109     sns.heatmap(cm, annot=True, fmt='g', cmap='Blues', xticklabels=['bus', 'cat', 'pizza
110         '], yticklabels=['bus', 'cat', 'pizza'])
111     plt.xlabel('Predicted_Labels')
112     plt.ylabel('True_Labels')
113     plt.title('Confusion_Matrix')
114     plt.savefig('Confusion_Matrix')
115     for i in range(3): # Print classification accuracies
116         print(f'Class_prediction_accuracy_for_class_{class_list[i]}:{class_correct[i]/
117             class_total[i]}')
118
119 def display_img(disDataset, gt_bboxes, gt_classes, pred_bboxes, pred_classes, img_idx,
120     root):
121     img_PIL, _ = disDataset[img_idx]
122     img = np.array(img_PIL)
123     for i in range(len(gt_classes)):
124         gt_bbox = gt_bboxes[i]
125         cv2.rectangle(img, (int(gt_bbox[0]), int(gt_bbox[1])), (int(gt_bbox[2]), int(
126             gt_bbox[3])), (0, 255, 0), 2)
127         cv2.putText(img, f'{gt_classes[i]}', (int(gt_bbox[0]), int(gt_bbox[1])), cv2.
128             FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
129     for i in range(len(pred_classes)):
130         pred_bbox = pred_bboxes[i]
131         cv2.rectangle(img, (int(pred_bbox[0]), int(pred_bbox[1])), (int(pred_bbox[2]),
132             int(pred_bbox[3])), (0, 0, 255), 2)
133         cv2.putText(img, f'{pred_classes[i]}', (int(pred_bbox[0]), int(pred_bbox[1])),
134             cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
135     if not os.path.exists(root):
136         os.makedirs(root)
137     cv2.imwrite(f'{root}/output{img_idx}.jpg', img)
138
139 def get_bbox(indices, yolo_vector):
140     x_grid = indices[1]
141     y_grid = indices[0]
142     x_diff = yolo_vector[1]
143     y_diff = yolo_vector[2]
144     h = yolo_vector[3]*256
145     w = yolo_vector[4]*256
146     x0 = ((x_grid + 0.5)*32/8 + x_diff)*8 # center of bounding box
147     y0 = ((y_grid + 0.5)*32/8 + y_diff)*8
148     return [x0-w/2, y0-h/2, x0+w/2, y0+h/2]
149
150 def IoU_calculator(bbox_gt, bbox_pred):
151     total_intersection = 0
152     total_union = 0
153     for i in range(len(bbox_pred)): # Iterate over all predicted bboxes
154         pred_bbox = bbox_pred[i]

```

```

143     pred_x1, pred_y1, pred_x2, pred_y2 = pred_bbox
144     max_iou = 0 # Initialize max IoU for this predicted bbox
145     selected_intersection = 0
146     selected_union = 0
147     for j in range(len(bbox_gt)): # Iterate over all ground truth bboxes
148         gt_bbox = bbox_gt[j]
149         gt_x1, gt_y1, gt_x2, gt_y2 = gt_bbox
150         # Calculate area of both boxes
151         pred_area = (pred_x2 - pred_x1) * (pred_y2 - pred_y1)
152         gt_area = (gt_x2 - gt_x1) * (gt_y2 - gt_y1)
153         inter_x1 = max(pred_x1, gt_x1)
154         inter_y1 = max(pred_y1, gt_y1)
155         inter_x2 = min(pred_x2, gt_x2)
156         inter_y2 = min(pred_y2, gt_y2)
157         if inter_x1 >= inter_x2 or inter_y1 >= inter_y2: # check that bounding boxes
158             intersect
159             intersection = 0 # no intersection
160         else:
161             intersection = (inter_x2 - inter_x1) * (inter_y2 - inter_y1)
162         union = pred_area + gt_area - intersection
163         iou = intersection / union
164         if iou >= max_iou: # Keep track of the best match
165             # only update intersection if iou improves
166             max_iou = iou
167             selected_intersection = intersection
168             selected_union = union
169         # Add the highest IoU for the current predicted bbox to total IoU
170         total_intersection += selected_intersection
171         total_union += selected_union
172     return total_intersection / total_union

```

5.2 Code Description

Lines 127 to 136 of the code above takes a yolo vector and the indices of its position in the entire $8 \times 8 \times 5 \times 8$ yolo tensor of the image to calculate the bbox coordinates. From the indices, the x and y grid can be found. From the yolo vector, the displacements of the center of the bbox to the center of the grid cell can be extracted (x_diff, y_diff). Then, the center of the bounding box can be calculated using the grid center coordinates and the displacement coordinates. Next, the height and width of the bbox is extracted from the yolo vector. With the center of the bounding box and the height and width of the bounding box, the upper left and lower right corner coordinates can be calculated.

The class labels are found by simply finding the maximum of the final three positions of the yolo vectors.

5.3 Good Single Instance Examples

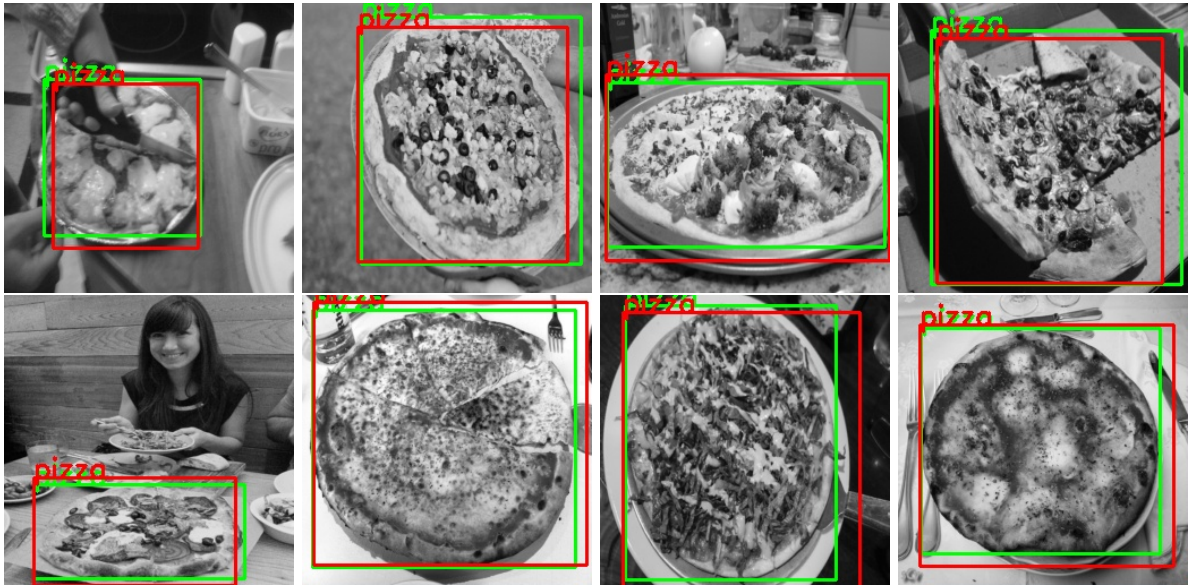


Figure 7: fig: Good Detections: Pizza

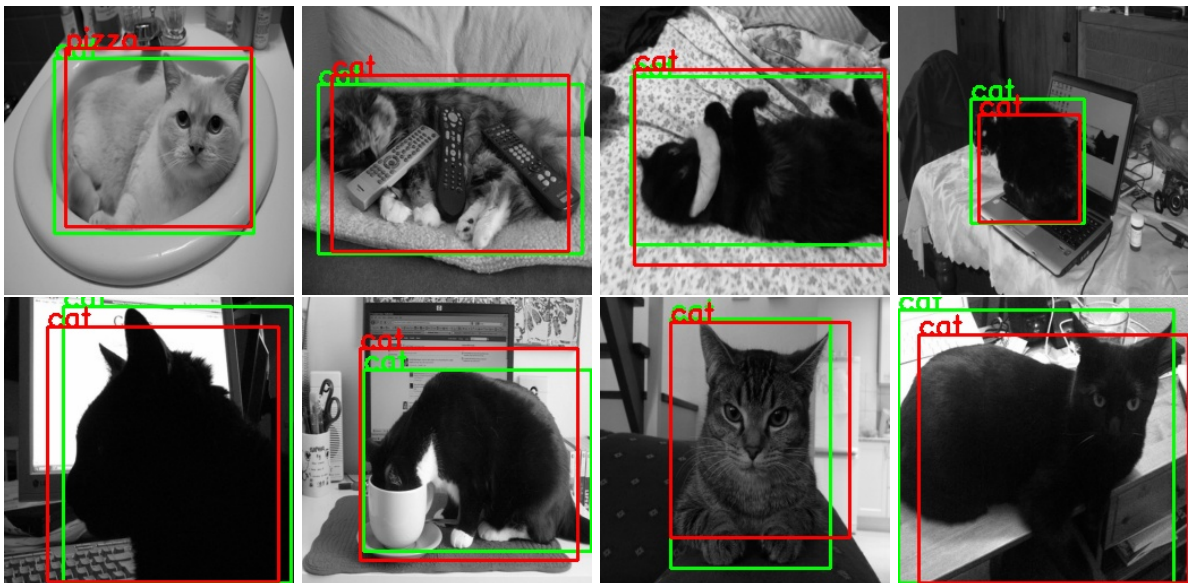


Figure 8: fig: Good Detections: Cat

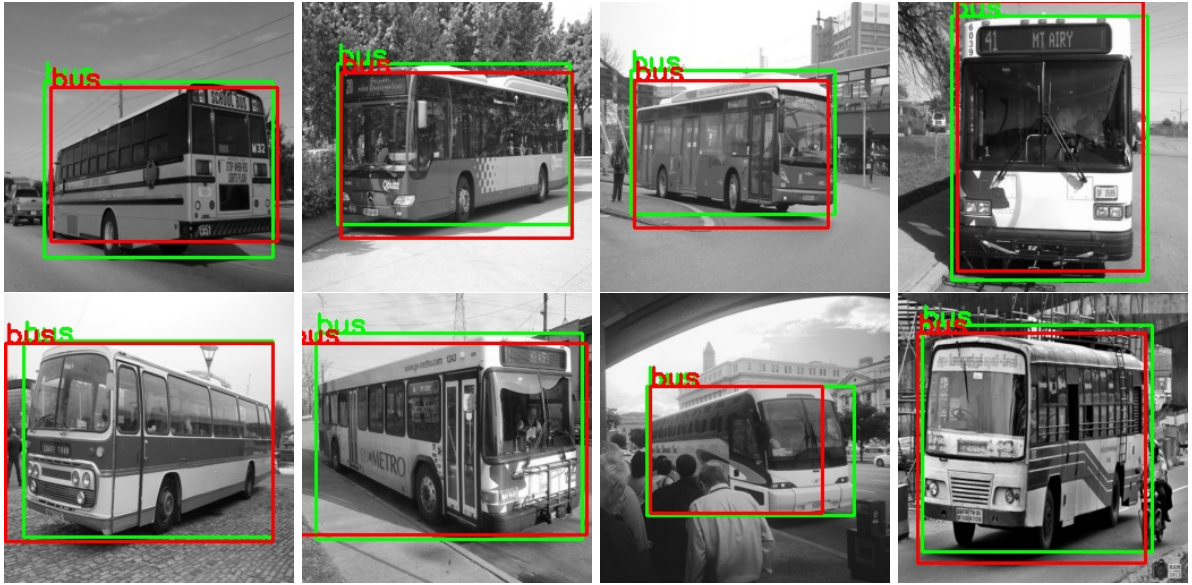


Figure 9: fig: Good Detections: Bus

5.4 Bad Single Instance Examples

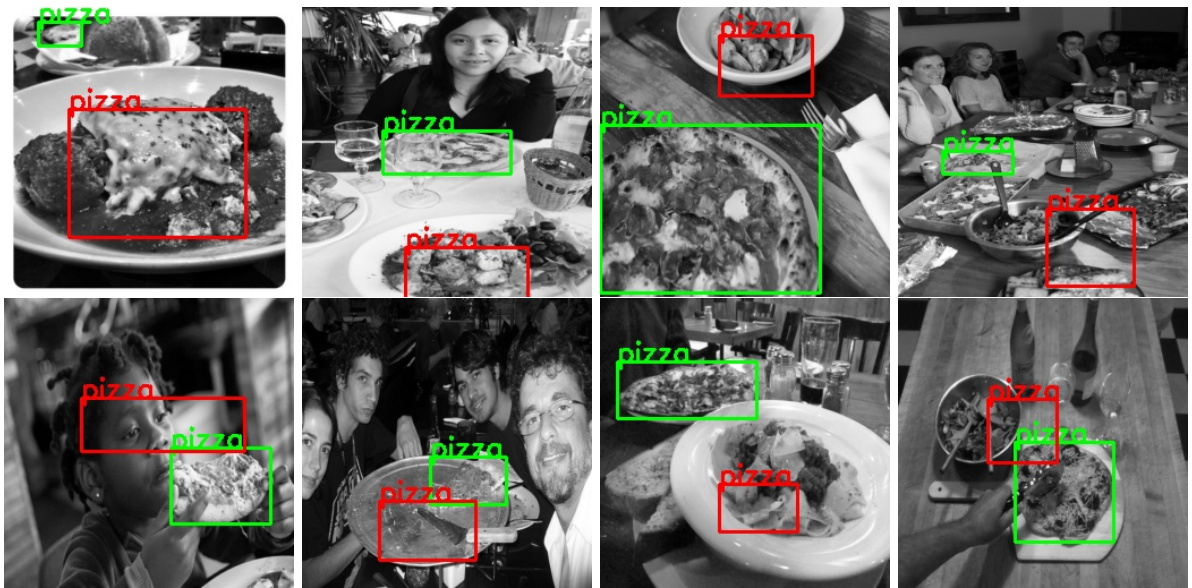


Figure 10: fig: Bad Detections: Pizza



Figure 11: fig: Bad Detections: Cat

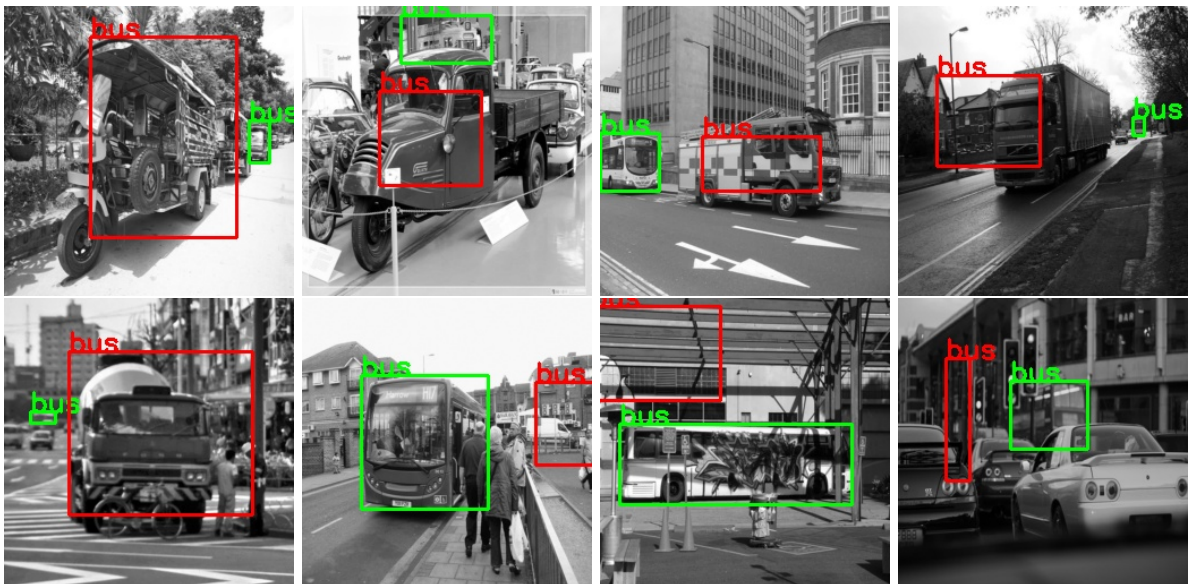


Figure 12: fig: Bad Detections: Bus

5.5 Good Multi Instance Examples

In the original dataset, there were some pizza examples where the same pizza had two bounding boxes on it. My network was able to predict just one bounding box.

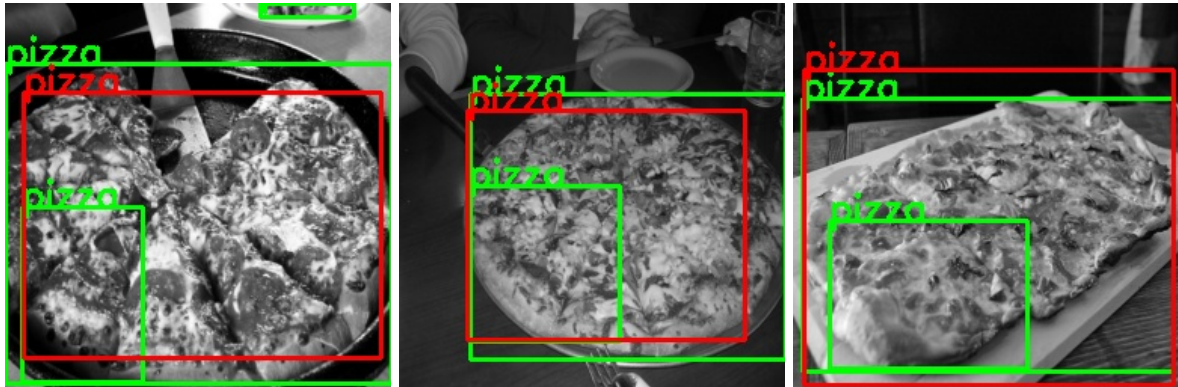


Figure 13: fig: Good Multi Detections: Pizza

For multi-instance images of cats, my model was unable to detect multiple instances. Therefore, detection of multi-instance cat images were deemed "good" if the network could segment out one a single cat instance.

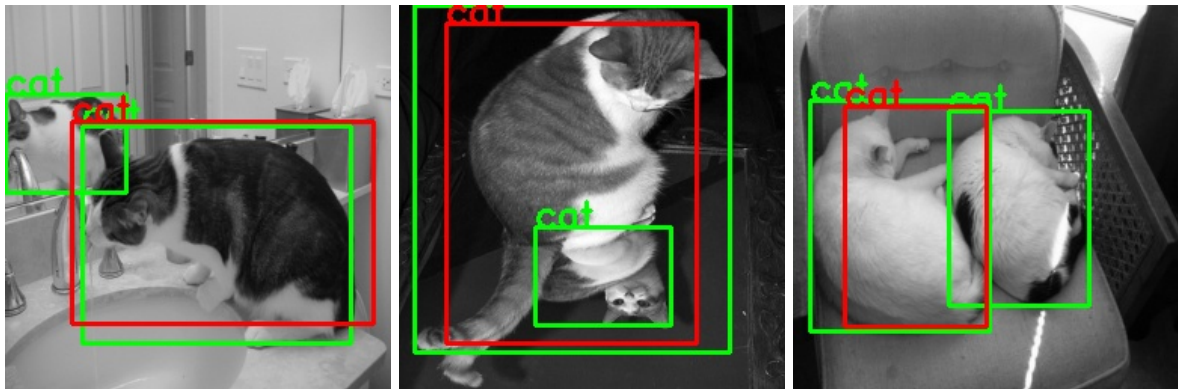


Figure 14: fig: Good Multi Detections: Cat

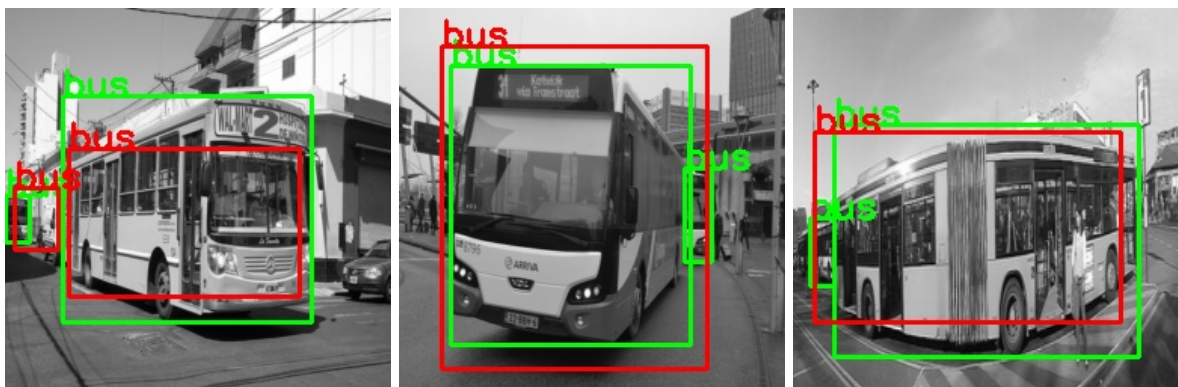


Figure 15: fig: Good Multi Detections: Bus

5.6 Bad Multi Instance Examples

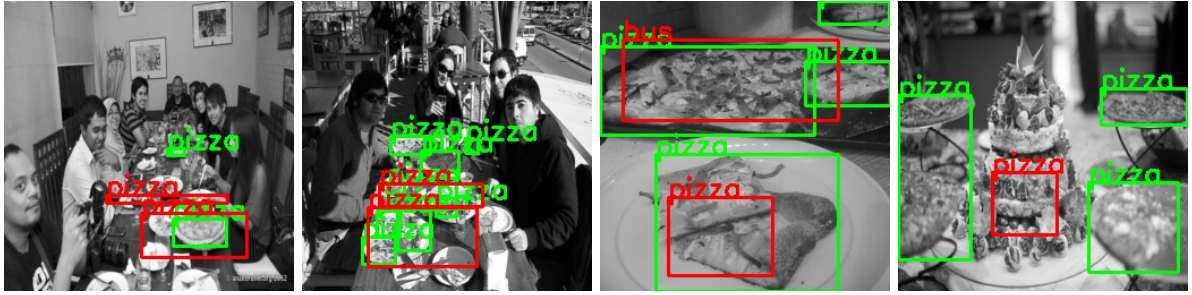


Figure 16: fig: Bad Multi Detections: Pizza

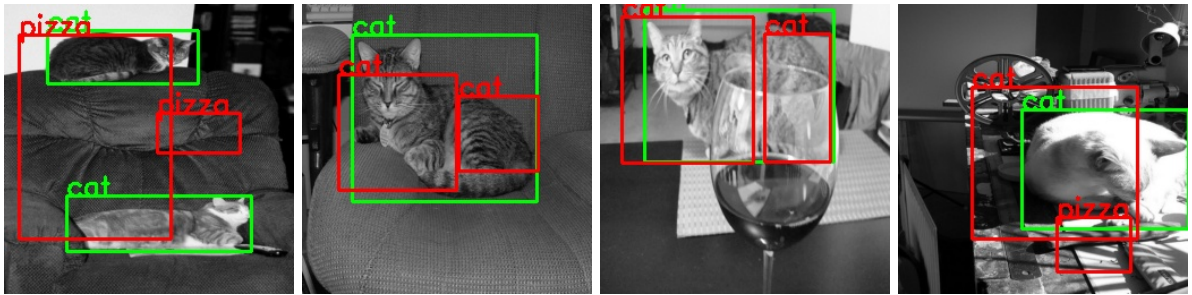


Figure 17: fig: Bad Multi Detections: Cat

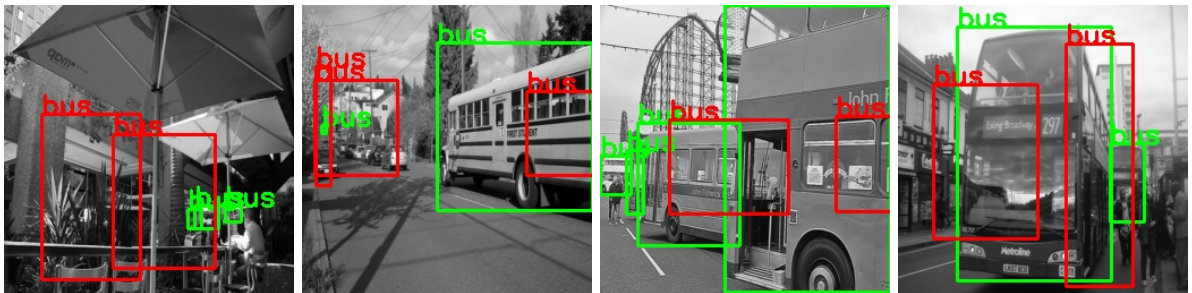


Figure 18: fig: Bad Multi Detections: Bus